

NexusLedger

Core Engine para Processamento Financeiro e Gestão de Contas Bancárias

Resumo Executivo

Este documento apresenta a especificação arquitetural, a modelagem de dados e os requisitos técnicos do **NexusLedger**. O artefato descreve uma solução transacional rigorosa, construída fundamentalmente sobre os pilares da Programação Orientada a Objetos (POO). O projeto foca em evidenciar o domínio prático sobre abstração, herança, polimorfismo, encapsulamento defensivo e proteção de estado em memória. A aplicação utiliza recursos modernos da linguagem Java (como *records*), implementa orquestração de serviços com travas de segurança rigorosas, tratamento customizado de exceções financeiras e valida a lógica de negócios por meio de uma suíte de testes unitários automatizados.

Autor Principal:	Luís Gustavo Silva
Função:	Desenvolvedor de Software Java
Stack Core:	Java 17+, Maven, JUnit 5
Paradigma Base:	Programação Orientada a Objetos (POO)
Ambiente de Dev:	IntelliJ IDEA
Versão do Documento:	1.0.0 (MVP Core Banking)
Data de Publicação:	17 de agosto de 2026

Sumário

1	Visão Geral e Escopo do Projeto	2
1.1	Problema de Negócio	2
1.2	Stack Tecnológica	2
1.3	Objetivos Técnicos de Engenharia	2
2	Metodologia Ágil Adaptada	3
2.1	Papéis Acumulados (Solo Developer)	3
2.2	CrITÉrios de Aceite e Definition of Done (DoD)	3
3	Metodologia e Arquitetura do Sistema	4
3.1	Módulos da Aplicação	4
3.2	Requisitos de Usuário e Sistema	5
3.2.1	Visão do Usuário (Negócio)	5
3.2.2	Visão de Sistema (Engenharia)	6
4	Implementação Técnica e Paradigmas POO	7
4.1	Abstração e Encapsulamento de Estado	7
4.2	Integração com Coleções e Blindagem de Memória	7
4.3	Herança, Overriding e Operador Ternário	8
4.4	Polimorfismo e Contratos Obrigatórios	8
4.5	Sobrecarga de Métodos (Method Overloading)	9
4.6	Imutabilidade Estrutural com Java Records	9
5	Orquestração de Lógica Complexa e Sessão	10
5.1	BancoService: O Banco de Dados em Memória	10
5.2	App Main: Gerenciamento de Sessão e Primary Key	10
6	Tratamento de Exceções e Resiliência (Fail-Fast)	11
6.1	Exceções Customizadas de Domínio (Unchecked)	11
7	Garantia de Qualidade com Testes Unitários	12
7.1	Isolamento de Cenários e Asserções	12
7.2	Prova de Blindagem de Memória	12
8	Modelagem Visual e Diagramação UML	13
8.1	Diagrama de Classes (Estrutura e POO)	13
8.2	Diagrama de Casos de Uso (Visão de Requisitos)	14
8.3	Diagrama de Atividades (Fluxo Transacional)	15
9	Repositório e Versionamento	16
10	Conclusão	17

1 Visão Geral e Escopo do Projeto

O **NexusLedger** é um motor de processamento financeiro (back-end core) projetado para simular e gerenciar as operações fundamentais de uma instituição bancária. O projeto foca no rigor técnico, aplicação prática de lógicas de negócio estruturadas, regras de validação consistentes e cobertura abrangente de testes unitários.

1.1 Problema de Negócio

No ecossistema de serviços financeiros, a consistência dos dados e a rastreabilidade das transações são vitais. O desafio principal deste projeto consiste em projetar uma arquitetura capaz de processar transações entre contas bancárias com naturezas distintas (Corrente com limite e Poupança com rendimento), aplicando travas rigorosas que impeçam estados ilegais na memória (como transações nulas, auto-transferências ou saques a descoberto não autorizados).

1.2 Stack Tecnológica

Linguagem Core: Java 17+ (aproveitando *record types* para DTOs imutáveis).

Build Tool: Maven para gerenciamento de ciclo de vida e dependências.

IDE: IntelliJ IDEA.

Framework de Teste: JUnit 5 para garantia de qualidade e isolamento de cenários (BDD).

Modelagem Visual: Astah UML para especificação de requisitos e estrutura de objetos.

1.3 Objetivos Técnicos de Engenharia

1. **Consistência e Segurança OO:** Aplicar rigorosamente o encapsulamento defensivo e o lançamento de exceções customizadas (`SaldoInsuficienteException`, `ValorInvalidoException`) para garantir a integridade dos saldos.
2. **Imutabilidade do Ledger:** Garantir que os registros de transações não possam ser adulterados após sua criação, simulando a segurança de um Livro-Razão autêntico.
3. **Orquestração de Regras Complexas:** Centralizar lógicas transacionais (como o PIX) em uma camada de serviço (`BancoService`) que atue como validador primário antes da manipulação dos objetos de domínio.

2 Metodologia Ágil Adaptada

Para conferir ritmo, consistência e simular de forma realista um ambiente de desenvolvimento corporativo, o projeto foi gerido sob um modelo simplificado focado em entregas de valor contínuo.

2.1 Papéis Acumulados (Solo Developer)

Para fins de rastreabilidade técnica e organizativa, os seguintes papéis são exercidos pelo autor:

Arquiteto de Software: Define a modelagem estrutural em UML, os limites de cada classe (SRP) e o ecossistema de tratamento de exceções.

Developer: Implementa as features utilizando o paradigma Orientado a Objetos e constrói o arcabouço de testes unitários.

2.2 Critérios de Aceite e Definition of Done (DoD)

Nenhum requisito técnico é considerado finalizado (“Pronto”) se não atender cumulativamente aos seguintes pontos:

O código compila sem warnings críticos ou erros locais.

Os testes unitários das regras core (`ContaCorrenteTest`, `ContaPoupancaTest`) passam integralmente.

Validações críticas (como saque acima do limite) devem disparar as exceções apropriadas.

O modelo UML (Casos de Uso, Diagrama de Classes e Atividades) foi atualizado para refletir o comportamento do código-fonte.

3 Metodologia e Arquitetura do Sistema

O desenvolvimento foi orientado pela entrega de um Produto Mínimo Viável (MVP) do núcleo financeiro (*Core Banking*), estruturando a aplicação em pacotes lógicos (**core**, **service**, **exception**). A arquitetura garante que entidades de domínio possuam lógicas ricas e que a camada de serviço orquestre operações que envolvam múltiplas entidades simultaneamente.

3.1 Módulos da Aplicação

A arquitetura do sistema foi decomposta em módulos principais para garantir a separação de responsabilidades e facilitar a evolução do código, apresentados na Tabela 1.

Tabela 1: Módulos do Sistema NexusLedger

ID	Módulo	Descrição do Escopo	Atores Envolvidos
M1	Domínio de Contas	Estruturação de base e herança das contas bancárias (ContaCorrente e ContaPoupanca), encapsulando regras estritas de saldo e limite.	Cliente
M2	Orquestração Transacional	Responsabilidade do BancoService . Gerencia operações atômicas entre contas distintas, aplicando travas lógicas para PIX (bloqueio de Auto-PIX e validação de existência).	Cliente, Serviço Central
M3	Motor Polimórfico	Cálculos dinâmicos delegados diretamente às instâncias (rendimentos de poupança e cobrança de tributos baseada no tipo da conta).	Serviço Central
M4	Auditoria e Ledger	Registro de operações com descrições customizadas geradas via sobrecarga de métodos e armazenamento protegido (unmodifiableList).	Cliente

3.2 Requisitos de Usuário e Sistema

A especificação de requisitos traduz as necessidades de negócio em funcionalidades tangíveis e restrições arquiteturais. Eles estão divididos em Funcionais (ações) e Não Funcionais (qualidade e restrições), tanto na perspectiva do usuário quanto da engenharia.

3.2.1 Visão do Usuário (Negócio)

Os requisitos de usuário descrevem as interações e expectativas do cliente bancário com a plataforma.

Tabela 2: Requisitos Funcionais de Usuário

ID	Módulo	Descrição
RUF01	Contas	O cliente deve possuir instâncias de Conta Corrente ou Poupança, identificadas por número e agência.
RUF02	Transações	O cliente deve realizar depósitos recebendo validação contra valores negativos ou zerados.
RUF03	Transações	O cliente deve realizar saques, sendo o limite de aprovação flexível na Corrente (Cheque Especial) e restrito ao saldo na Poupança.
RUF04	Transações	O cliente deve transferir valores para terceiros via PIX de forma integrada.
RUF05	Auditoria	O cliente deve consultar o histórico de transações e a descrição da operação (ex: "PIX ENVIADO").

Tabela 3: Requisitos Não Funcionais de Usuário

ID	Categoria	Descrição
RUNF01	Confiabilidade	Operações inválidas não podem comprometer o saldo em nenhuma hipótese; mensagens de erro de regras de negócio devem ser claras.
RUNF02	Transparência	O extrato financeiro deve avisar explicitamente (visualmente na descrição) quando o cliente utilizar o limite do Cheque Especial.

3.2.2 Visão de Sistema (Engenharia)

Os requisitos de sistema traduzem o negócio em regras estritas de modelagem Orientada a Objetos, definindo como o código deve se comportar estruturalmente.

Tabela 4: Requisitos Funcionais de Sistema

ID	Domínio	Descrição Técnica
RSF01	OO & Herança	A classe <code>ContaBancaria</code> deve ser <i>abstract</i> , fornecendo o esqueleto estrutural (atributos e lista) para suas ramificações.
RSF02	Imutabilidade	A entidade <code>Transacao</code> deve ser implementada como um Java <i>Record</i> , garantindo a natureza <i>final</i> dos seus campos após o registro.
RSF03	Orquestração	O método <code>realizarPix()</code> deve validar se origem e destino são iguais e se as contas existem na lista gerenciada pelo <code>BancoService</code> .
RSF04	Polimorfismo	O cálculo de tributos deve ser despachado via <i>override</i> (0.0 para poupança e 1% do saldo para corrente).

Tabela 5: Requisitos Não Funcionais de Sistema

ID	Categoria	Descrição Técnica
RSNF01	Segurança de Memória	A lista de histórico de transações deve ser exposta apenas como uma <code>unmodifiableList</code> , blindando o sistema contra limpezas indevidas (<code>.clear()</code>).
RSNF02	Testabilidade	As regras vitais de negócio e tratamento de exceções devem possuir cobertura de testes unitários automatizados através do <i>JUnit 5</i> .
RSNF03	Tratamento Erros	Falhas de validação financeira devem obrigatoriamente quebrar o fluxo lançando exceções customizadas (<code>ValorInvalidoException</code> , <code>SaldoInsuficienteException</code>).

4 Implementação Técnica e Paradigmas POO

Este projeto foi intencionalmente arquitetado para demonstrar um domínio rigoroso sobre a lógica fundamental da linguagem Java e os pilares da Programação Orientada a Objetos (POO). A modelagem dispensou o uso de frameworks engessados para evidenciar a fluência na aplicação prática de abstração, encapsulamento, herança e polimorfismo.

4.1 Abstração e Encapsulamento de Estado

A classe `ContaBancaria` foi declarada como `abstract`. Esta decisão de design proíbe a instanciação de uma conta genérica (`new ContaBancaria()`), refletindo a regra de negócio do mundo real onde uma conta deve obrigatoriamente possuir uma especialização (Corrente ou Poupança).

O encapsulamento foi aplicado de forma agressiva: os atributos de identificação e saldo são privados (`private`) e não possuem *Setters*. A alteração do estado financeiro só ocorre mediante a passagem por fluxos validados (`depositar` e `sacar`), garantindo a integridade dos dados.

```
1  import java.util.ArrayList;
2  import java.util.Collections;
3  import java.util.List;
4
5  public abstract class ContaBancaria {
6      private String nmrConta;
7      private String agencia;
8      protected double saldo;
9      protected List<Transacao> historico;
10
11     public ContaBancaria(String nmrConta, String agencia) {
12         this.nmrConta = nmrConta;
13         this.agencia = agencia;
14         // Previne NullPointerException instanciando a colecao no nascimento do
15         // objeto
16         this.historico = new ArrayList<>();
17     }
18
19     // Retorna uma view envelopada (somente leitura) da lista original
20     public List<Transacao> getHistorico() {
21         return Collections.unmodifiableList(historico);
22     }
23 }
```

Listing 1: Abstracao e encapsulamento na base do dominio

4.2 Integração com Coleções e Blindagem de Memória

Como evidenciado no código acima, o sistema faz uso maduro da *Java Collections Framework*. O histórico é declarado pela interface `List` (desacoplamento) e inicializado no construtor como `ArrayList`.

Para atender aos rigorosos requisitos de segurança bancária, o método `getHistorico()` não devolve a referência direta da lista, mas sim um envelope criado por `Collections.unmodifiableList`. Isso impede completamente que sistemas externos injetem transações falsas (`.add()`) ou apaguem rastros de auditoria (`.clear()`).

4.3 Herança, Overriding e Operador Ternário

As classes filhas (`ContaCorrente` e `ContaPoupanca`) aplicam a Herança (`extends`) para reaproveitar atributos base, acionando o construtor da superclasse via `super()`.

A `ContaCorrente` especializa a regra de negócio sobrescrevendo (`@Override`) o método `sacar()`, expandindo a validação para considerar o `limiteChequeES`. Uma inteligência de auditoria foi adicionada: caso a operação negative o cliente, o sistema utiliza um ****Operador Ternário**** para concatenar dinamicamente o aviso de uso do Cheque Especial na descrição do extrato.

```
1 public class ContaCorrente extends ContaBancaria {
2     private double limiteChequeES;
3
4     public ContaCorrente(String nmrConta, String agencia, double limite) {
5         super(nmrConta, agencia); // Reaproveitamento de código da classe mae
6         this.limiteChequeES = limite;
7     }
8
9     @Override
10    public void sacar(double valor, String descricao){
11        if(valor <= 0){
12            throw new ValorInvalidoException("O valor do saque deve ser maior que 0.");
13        } else if(valor > this.saldo + this.limiteChequeES){
14            throw new SaldoInsuficienteException("Saldo Insuficiente. Limite excedido.");
15        } else {
16            this.saldo -= valor;
17
18            // Logica inline para auditoria dinamica
19            String descFinal = (this.saldo < 0) ? descricao + " (USO DO CHEQUE ESPECIAL)" : descricao;
20
21            Transacao t = new Transacao(UUID.randomUUID(), LocalDateTime.now(), descFinal, valor);
22            this.historico.add(t);
23        }
24    }
25 }
```

Listing 2: Sobrescrita de método e inteligência de negócio via Operador Ternário

4.4 Polimorfismo e Contratos Obrigatórios

O sistema utiliza Polimorfismo puro através do método abstrato `calcularTributos()` na classe mãe. Isso cria um contrato obrigatório: sem a implementação dessa fórmula, as classes filhas não compilam. Esse conceito elimina a necessidade de cadeias de `if/else` (*anti-pattern*) na camada de serviço para verificar o tipo da conta na hora de cobrar taxas, pois a JVM executa o cálculo correto em tempo de execução.

Nesta regra de negócio, a `ContaCorrente` aplica uma tributação de 1% sobre o saldo (`this.saldo * 0.01`), enquanto a `ContaPoupanca` garante sua isenção fiscal (`retorna 0.0`). O lucro da Poupança foi separado dos tributos através de um comportamento exclusivo da sua especialização: o método `renderJuros()`, que aplica uma taxa percentual de rendimento e injeta o valor no saldo de forma rastreável.

4.5 Sobrecarga de Métodos (Method Overloading)

Para maximizar a flexibilidade do código, o conceito de Sobrecarga de Métodos foi aplicado. Foram criadas múltiplas assinaturas para as ações financeiras: um `depositar(valor)` padrão, que assume a descrição "DEPÓSITO", e um `depositar(valor, descricao)` especializado, que permite que integrações como o PIX enviem descrições ricas ("PIX RECEBIDO") para o extrato.

4.6 Imutabilidade Estrutural com Java Records

A representação do extrato não utiliza classes tradicionais (POJOs), mas adota os **records** do Java 14+. Um **record** é uma estrutura imutável por padrão: todos os campos nascem como **private final**. Isso converte a Transação em um autêntico DTO (*Data Transfer Object*) seguro, garantindo conformidade de auditoria onde uma transação jamais poderá ser modificada após sua criação.

```
1 package com.nexusledger.core;
2
3 import java.time.LocalDateTime;
4 import java.util.UUID;
5
6 // Cria uma classe final com campos privados finais e metodos gerados em tempo de
  compilacao
7 public record Transacao(
8     UUID id,
9     LocalDateTime timestamp,
10    String tipoOperacao,
11    double valor
12 ) {}
```

Listing 3: O DTO imutavel nativo para transporte de dados financeiros

5 Orquestração de Lógica Complexa e Sessão

Enquanto o pacote `core` retém o domínio, a arquitetura delega a coordenação interbancária e o fluxo de telas para camadas superiores (`service` e `app`), respeitando o Princípio da Responsabilidade Única (SRP - SOLID).

5.1 BancoService: O Banco de Dados em Memória

A classe `BancoService` gerencia uma lista viva (`List<ContaBancaria>`) que simula o banco de dados. A lógica de `buscarConta()` emprega um algoritmo de pesquisa sequencial padrão (laço `for`), que varre a memória e retorna a instância exata da conta para processamento.

No método `realizarPix`, o serviço orquestra as travas de segurança atômicas. Ele bloqueia tentativas de "Auto-PIX" (comparando origens e destinos) e atua como fiador da integridade do ecossistema, garantindo que o extrato não seja sujo por transações duplicadas ou inválidas.

```
1 public void realizarPix(String numeroOrigem, String numeroDestino, double valor)
2     {
3         // TRAVA 1: Auto-PIX
4         if (numeroOrigem.equals(numeroDestino)) {
5             throw new ValorInvalidoException("PIX Recusado: Nao transferir para a
6                 propria conta.");
7         }
8
9         ContaBancaria origem = buscarConta(numeroOrigem);
10        ContaBancaria destino = buscarConta(numeroDestino);
11
12        // TRAVA 2: Contas nulas
13        if (origem == null || destino == null) {
14            throw new ValorInvalidoException("PIX Recusado: Contas nao encontradas.")
15                ;
16        }
17
18        origem.sacar(valor, "PIX ENVIADO");
19        destino.depositar(valor, "PIX RECEBIDO");
20    }
```

Listing 4: Travas de segurança e orquestração no BancoService

5.2 App Main: Gerenciamento de Sessão e Primary Key

Na interface do usuário, a gestão de criação de contas simula uma Chave Primária (Primary Key) de banco de dados: antes de instanciar o objeto, o código verifica via condicional se a conta já existe, prevenindo o sobre-cadastro. O controle de sessão é feito através de uma referência inteligente (`contaLogada`), que direciona o fluxo de menus baseada na sua nulidade.

6 Tratamento de Exceções e Resiliência (Fail-Fast)

Sistemas críticos não podem depender de códigos de erro amadores (como retornar `false` ou `-1`). O **NexusLedger** aplica o princípio arquitetural *Fail-Fast*: a execução explode e para imediatamente ao identificar uma regra quebrada.

6.1 Exceções Customizadas de Domínio (Unchecked)

Foram projetadas classes como `SaldoInsuficienteException` que estendem `RuntimeException`. O construtor repassa a responsabilidade para a superclasse via `super(mensagem)`. Isso remove a poluição de cláusulas `throws` no código e garante que o usuário receba o aviso exato de qual regra de negócio foi violada, evitando crashes genéricos e complexos de rastrear.

```
1 package com.nexusledger.exception;
2
3 public class SaldoInsuficienteException extends RuntimeException {
4     public SaldoInsuficienteException(String mensagem) {
5         super(mensagem); // Repassa o erro de domínio para a JVM
6     }
7 }
```

Listing 5: Arquitetura de Excecoes Customizadas

7 Garantia de Qualidade com Testes Unitários

O projeto possui uma malha fina de *Quality Assurance* orientada a comportamento (BDD). Robôs de teste escritos em **JUnit 5** provam matematicamente que o código fonte obedece rigorosamente as regras da documentação.

7.1 Isolamento de Cenários e Asserções

Através da anotação `@BeforeEach`, o ambiente é reidratado (uma conta limpa é criada) antes de cada cenário para impedir a "sujeira de estado" entre os testes. Cenários críticos utilizam `assertEquals` para validar que a lógica matemática abate e credita com precisão centesimal. O relatório de execução foi traduzido para linguagem humana usando a anotação `@DisplayName`.

7.2 Prova de Blindagem de Memória

Uma arquitetura forte se prova testando o fracasso das invasões. A suíte utiliza o `assertThrows` para verificar se o sistema reage agressivamente a fraudes lógicas. O código abaixo comprova, rodando a JVM, que a tentativa de limpar os rastros financeiros ou sacar além do limite engatilham instantaneamente a defesa do sistema.

```
1 import org.junit.jupiter.api.DisplayName;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 @Test
6 @DisplayName("Deve bloquear saque que ultrapasse o saldo mais o limite")
7 void testBloquearSaqueAcimaDoLimite() {
8     SaldoInsuficienteException erro = assertThrows(SaldoInsuficienteException.class, () -> {
9         conta.sacar(600.00);
10    });
11
12    assertEquals("Saldo Insuficiente. Limite excedido.", erro.getMessage());
13 }
14
15 @Test
16 @DisplayName("Deve impedir modificacao externa na lista de historico (Imutabilidade)")
17 void testImpedirLimpezaDoHistorico() {
18     conta.depositar(100.00);
19
20     // O teste garante que fraudar o ledger gerara UnsupportedOperationException
21     assertThrows(UnsupportedOperationException.class, () -> {
22         conta.getHistorico().clear();
23     });
24 }
```

Listing 6: Assercoes avancadas para seguranca de regras e memoria

8 Modelagem Visual e Diagramação UML

Para garantir que a implementação das regras de negócio mantivesse uma arquitetura coesa e rastreável, o sistema foi previamente modelado utilizando a linguagem UML (Unified Modeling Language). Os diagramas abaixo ilustram as visões de requisitos, estrutura orientada a objetos e fluxo de execução.

8.1 Diagrama de Classes (Estrutura e POO)

O Diagrama de Classes representa o coração do paradigma Orientado a Objetos do projeto. Ele mapeia a herança a partir da classe abstrata **ContaBancaria**, evidenciando os níveis de visibilidade dos atributos (**private** e **protected**) e a relação de composição imutável com o *record* de **Transacao**. A orquestração é representada pela agregação no **BancoService**.

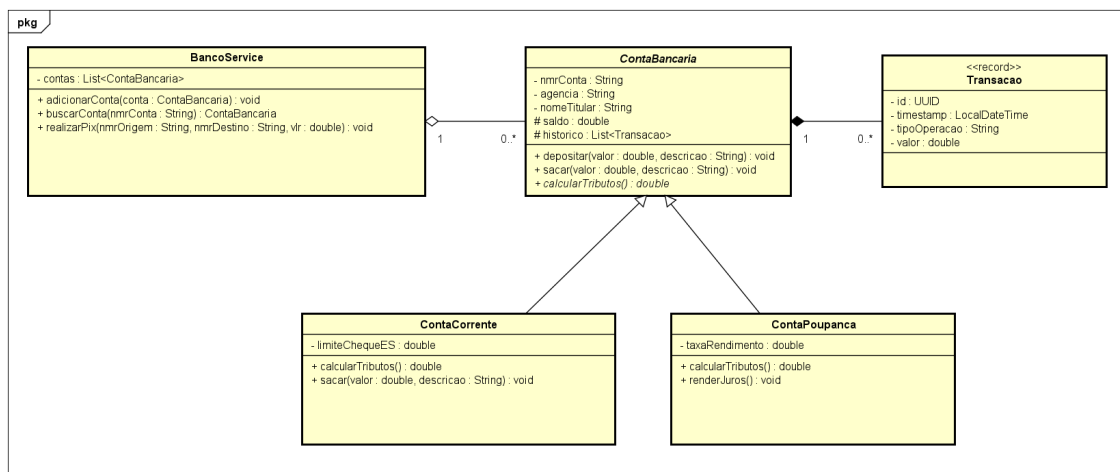


Figura 1: Diagrama de Classes evidenciando herança, agregação e composição.

Fonte: Autoria própria.

8.2 Diagrama de Casos de Uso (Visão de Requisitos)

O Diagrama de Casos de Uso detalha as interações do **Cliente** com o sistema. Observa-se a utilização rigorosa dos relacionamentos `<<include>>` (tornando o Login um ponto de passagem obrigatório para todas as operações transacionais) e o uso da extensão opcional `<<extend>>` para o acionamento dinâmico do Cheque Especial durante saques e transferências.

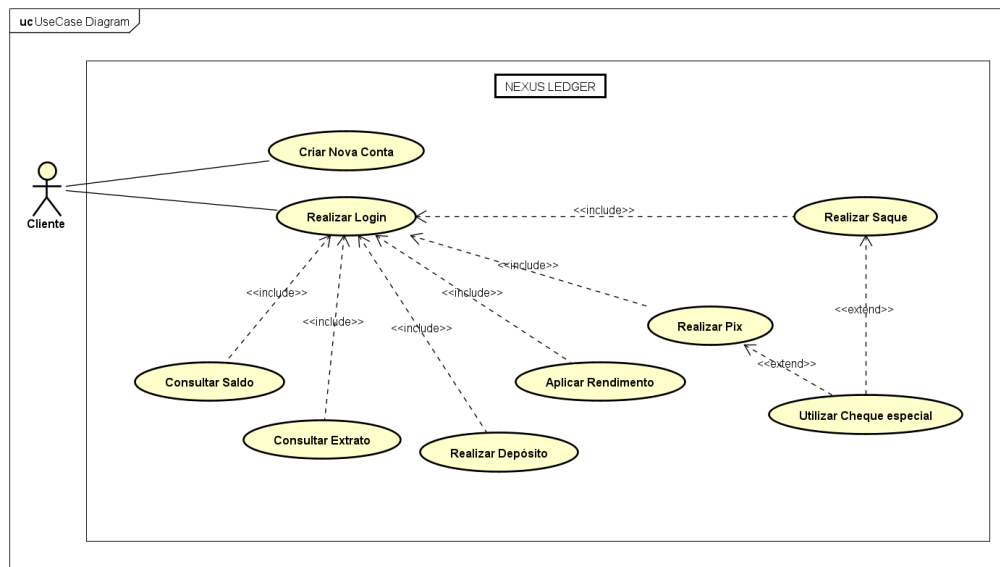


Figura 2: Diagrama de Casos de Uso demonstrando fluxos obrigatórios e opcionais.

Fonte: Autoria própria.

8.3 Diagrama de Atividades (Fluxo Transacional)

Este diagrama reflete as ramificações lógicas (**if/else**) do método **realizarPix()** da camada de serviço. O mapeamento ilustra a sequência de nós de decisão (losangos) atuando como travas de validação sequenciais (verificação de conta destino, bloqueio de auto-PIX e saldo disponível) antes de permitir a alteração atômica dos saldos.

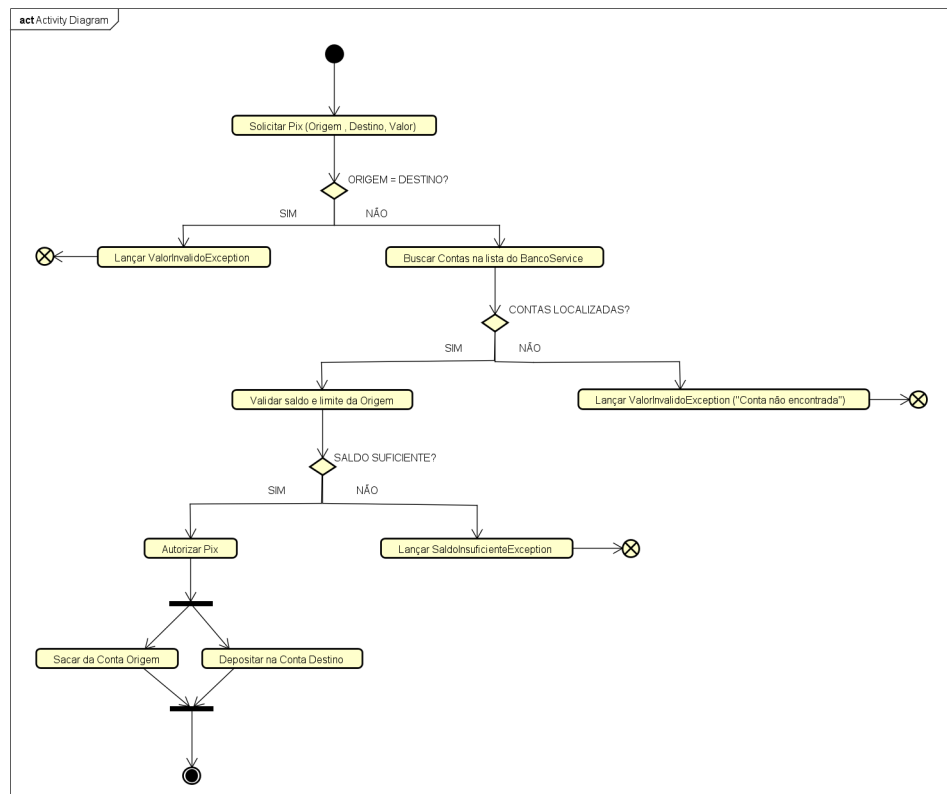


Figura 3: Fluxo de decisão detalhando as travas de validação do PIX.

Fonte: Autoria própria.

9 Repositório e Versionamento

O projeto foi inteiramente versionado localmente e distribuído através da plataforma GitHub, seguindo padrões semânticos de commit e permitindo o rastreamento contínuo da evolução arquitetural e da inclusão da suíte de testes.

O código-fonte completo, acompanhado das instruções para execução e rodada de testes via Maven, pode ser inspecionado publicamente no repositório abaixo:

Acesso ao Código-Fonte (GitHub):

<https://github.com/luigustavos29/NexusLedger.git>

10 Conclusão

O desenvolvimento do **NexusLedger** evidenciou que a verdadeira solidez de um motor transacional não provém do uso prematuro de frameworks complexos, mas sim do domínio absoluto sobre a lógica de programação e os pilares fundamentais da Orientação a Objetos.

A decisão arquitetural de focar na base do ecossistema Java aplicando abstração rigorosa, encapsulamento defensivo de coleções para proteção de estado e polimorfismo para erradicar ramificações condicionais desnecessárias gerou um núcleo de negócios altamente coeso e imune a acoplamentos nocivos. Somado a isso, a cobertura analítica via JUnit 5 não apenas validou os cenários ideais, mas comprovou matematicamente a resiliência sistêmica contra fraudes lógicas e violações de memória (Princípio *Fail-Fast*).

Mais do que um simulador bancário corporativo, este projeto consolida-se como um artefato técnico definitivo em formato de portfólio. Ele atesta uma maturidade abrangente em Engenharia de Software, comprovando a capacidade real de levantar requisitos, modelar arquiteturas rastreáveis via UML e codificar soluções escaláveis pautadas nas melhores práticas e padrões de projeto da linguagem Java.